Assignment - 7

Course: MST501-1: Python Programming for Statistics

Done by:

Name: Ankita Das

Register Number: 2648103

Date: 30/07/2026

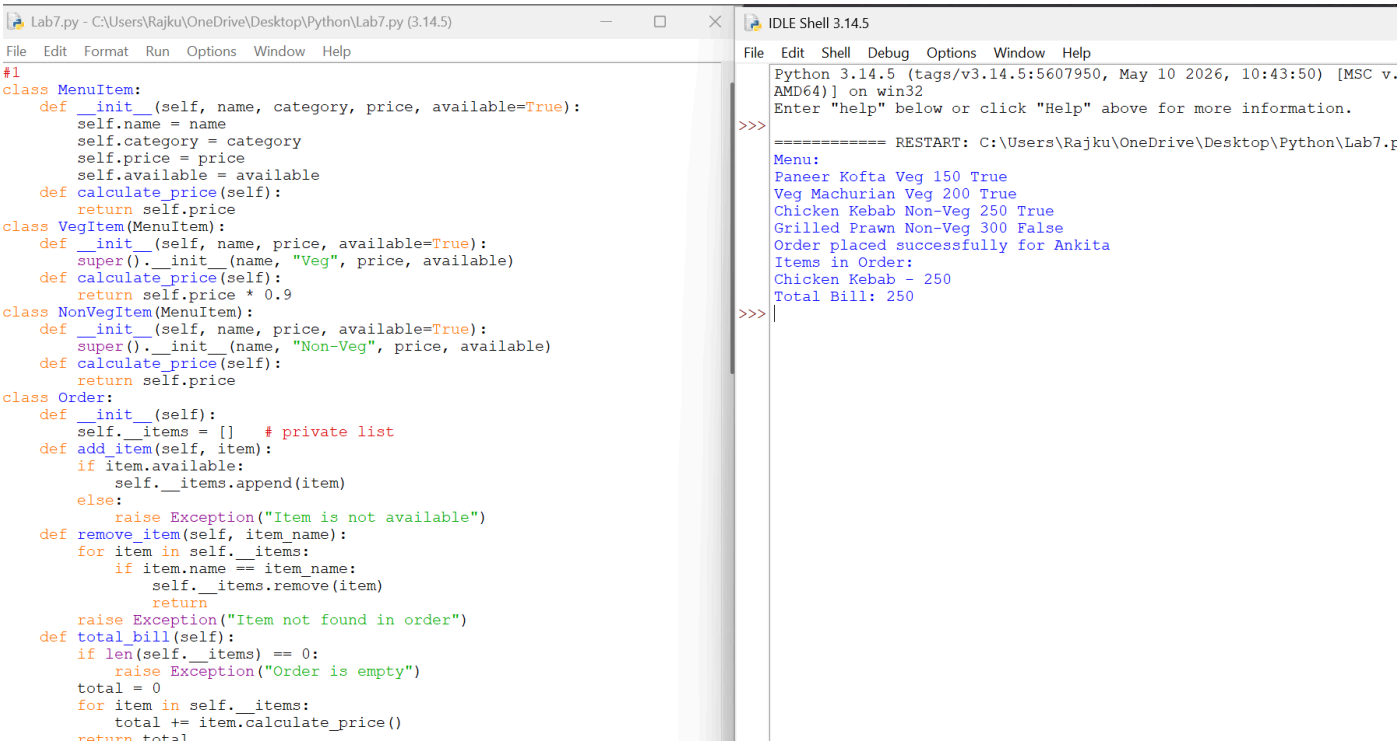
1. Develop a Python application for a simple food ordering platform. Create classes such as MenuItem, Order, Customer, and Restaurant. Each menu item should include attributes like name, category, price, and availability. Customers should be able to add items to an order, remove items, calculate the total bill, and place the order. The restaurant should maintain the menu and process customer orders. Use object-oriented principles by encapsulating order details, applying inheritance for different menu item categories (e.g., VegItem and NonVegItem), and implementing polymorphism for bill calculation if discounts or special offers are introduced. Include proper exception handling for invalid menu selections or empty orders.

Ans-

```
class MenuItem:
    def __init__(self, name, category, price, available=True):
        self.name = name
        self.category = category
        self.price = price
        self.available = available
    def calculate_price(self):
        return self.price
class VegItem(MenuItem):
    def __init__(self, name, price, available=True):
        super().__init__(name, "Veg", price, available)
    def calculate_price(self):
        return self.price * 0.9
class NonVegItem(MenuItem):
    def __init__(self, name, price, available=True):
        super().__init__(name, "Non-Veg", price, available)
    def calculate_price(self):
        return self.price
class Order:
    def __init__(self):
        self.__items = []   # private list
    def add_item(self, item):
        if item.available:
            self.__items.append(item)
        else:
            raise Exception("Item is not available")
    def remove_item(self, item_name):
        for item in self.__items:
            if item.name == item_name:
                self.__items.remove(item)
                return
        raise Exception("Item not found in order")
    def total_bill(self):
        if len(self.__items) == 0:
            raise Exception("Order is empty")
        total = 0
        for item in self.__items:
```

```
            total += item.calculate_price()
        return total
    def display_order(self):
        if len(self.__items) == 0:
            print("Order is empty")
        else:
            print("Items in Order:")
            for item in self.__items:
                print(item.name, "-", item.calculate_price())
    def is_empty(self):
        return len(self.__items) == 0
class Customer:
    def __init__(self, name):
        self.name = name
        self.order = Order()
    def add_to_order(self, restaurant, item_name):
        item = restaurant.find_item(item_name)
        self.order.add_item(item)
    def remove_from_order(self, item_name):
        self.order.remove_item(item_name)
    def place_order(self, restaurant):
        restaurant.process_order(self)
class Restaurant:
    def __init__(self, name):
        self.name = name
        self.menu = []
    def add_menu_item(self, item):
        self.menu.append(item)
    def show_menu(self):
        print("Menu:")
        for item in self.menu:
            print(item.name, item.category, item.price, item.available)
    def find_item(self, item_name):
        for item in self.menu:
            if item.name == item_name:
                return item
        raise Exception("Invalid menu selection")
    def process_order(self, customer):
        if customer.order.is_empty():
            raise Exception("Cannot place an empty order")
        print("Order placed successfully for", customer.name)
        customer.order.display_order()
        print("Total Bill:", customer.order.total_bill())
restaurant = Restaurant("Food Paradise")
restaurant.add_menu_item(VegItem("Paneer Kofta", 150))
restaurant.add_menu_item(VegItem("Veg Machurian", 200))
restaurant.add_menu_item(NonVegItem("Chicken Kebab", 250))
restaurant.add_menu_item(NonVegItem("Grilled Prawn", 300, False))
```

```
restaurant.show_menu()
customer = Customer("Ankita")
try:
    customer.add_to_order(restaurant, "Paneer Kofta")
    customer.add_to_order(restaurant, "Chicken Kebab")
    customer.remove_from_order("Paneer Kofta")
    customer.place_order(restaurant)
except Exception as e:
    print("Error:", e)
```



2. Create/ download a CSV file containing sales records with columns such as Order_ID, Product, Category, Quantity, Price, Salesperson, and Region. Write a Python program using the Pandas library to load the dataset, display the first and last five records, check the dataset's shape, column names, and data types, identify missing values, and generate summary statistics. Perform basic operations such as selecting specific columns, filtering records based on region or product category, calculating the total sales for each order (Quantity × Price), sorting the data by total sales, and saving the updated dataset to a new CSV file.

Ans-

```
import pandas as pd
df = pd.read_csv(r"C:\\Users\\Rajku\\OneDrive\\Desktop\\Python\\Datastore.csv")
print(df.head())
print(df.tail())
print(df.shape)
print(df.columns)
```

print(df.dtypes)
print(df.isnull().sum())
print(df.describe())
print("Product, Quantity and Price:")
print(df[["Product", "Quantity", "Price"]])
north = df[df["Region"] == "North"]
print(north)
electronics = df[df["Category"] == "Electronics"]
print(electronics)
df["Total_Sales"] = df["Quantity"] * df["Price"]
print("Dataset with Total Sales:")
print(df)
sorted_df = df.sort_values(by="Total_Sales", ascending=False)
print(sorted_df)
sorted_df.to_csv(r"C:\\Users\\Rajku\\OneDrive\\Desktop\\Python\\updated_sales_records.csv", index=False)
print("Updated dataset saved as updated_sales_records.csv")

```
#2
import pandas as pd
df = pd.read_csv(r"C:\\Users\\Rajku\\OneDrive\\Desktop\\Python\\Datastore.cs
print(df.head())
print(df.tail())
print(df.shape)
print(df.columns)
print(df.dtypes)
print(df.isnull().sum())
print(df.describe())
print("Product, Quantity and Price:")
print(df[["Product", "Quantity", "Price"]])
north = df[df["Region"] == "North"]
print(north)
electronics = df[df["Category"] == "Electronics"]
print(electronics)
df["Total_Sales"] = df["Quantity"] * df["Price"]
print("Dataset with Total Sales:")
print(df)
sorted_df = df.sort_values(by="Total_Sales", ascending=False)
print(sorted_df)
sorted_df.to_csv(r"C:\\Users\\Rajku\\OneDrive\\Desktop\\Python\\updated_sale
print("Updated dataset saved as updated_sales_records.csv")
```

```
   Order_ID Product     Category  Quantity  Price Salesperso
0       101  Laptop  Electronics         2  50000       Rahu
1       102   Mouse  Electronics         5    500       Anit
2       103   Chair    Furniture         3   3000      Vikra
3       104   Table    Furniture         1   7000       Priy
4       105   Phone  Electronics         4  20000       Rahu
   Order_ID   Product     Category  Quantity  Price Salesper
5       106      Sofa    Furniture         2  15000       An
6       107  Keyboard  Electronics         6   1000      Vik
7       108  Cupboard    Furniture         1  12000       Pr
8       109   Monitor  Electronics         3   8000       Ra
9       110       Bed    Furniture         1  25000       An
(10, 7)
Index(['Order_ID', 'Product', 'Category', 'Quantity', 'Price
       'Region'],
      dtype='str')
Order_ID       int64
Product          str
Category         str
Quantity       int64
Price          int64
Salesperson      str
Region           str
dtype: object
Order_ID       0
Product        0
Category       0
Quantity       0
Price          0
Salesperson    0
Region         0
dtype: int64
        Order_ID  Quantity         Price
count   10.00000  10.00000      10.000000
mean   105.50000   2.80000  14150.000000
std      3.02765   1.75119  14963.009947
min    101.00000   1.00000    500.000000
25%    103.25000   1.25000   4000.000000
50%    105.50000   2.50000  10000.000000
75%    107.75000   3.75000  18750.000000
```

3. Create/ download a CSV file containing student information with columns such as Student_ID, Name, Department, Math, Science, English, and Attendance. Using Pandas, write a program to read the dataset and perform basic data analysis. Display student records, calculate the average marks of each student, identify students scoring above a specified average, find the highest and lowest marks in each subject, sort students by average marks in descending order, and create a new column for grades based on average scores. Finally, export the modified DataFrame to a new CSV file

Ans- import pandas as pd

df = pd.read_csv(r"C:\\Users\\Rajku\\OneDrive\\Desktop\\Python\\student_records.csv")

```
print("Student Records:")
print(df)
df["Average"] = (df["Math"] + df["Science"] + df["English"]) / 3
print("Average Marks of Each Student:")
print(df[["Student_ID", "Name", "Average"]])
specified_average = 80
above_average = df[df["Average"] > specified_average]
print("Students Scoring Above", specified_average, "Average:")
print(above_average[["Student_ID", "Name", "Average"]])
print("Math:", df["Math"].max())
print("Science:", df["Science"].max())
print("English:", df["English"].max())
print("Math:", df["Math"].min())
print("Science:", df["Science"].min())
print("English:", df["English"].min())
sorted_df = df.sort_values(by="Average", ascending=False)
print(sorted_df[["Student_ID", "Name", "Average"]])
def calculate_grade(avg):
    if avg >= 90:
        return "A"
    elif avg >= 80:
        return "B"
    elif avg >= 70:
        return "C"
    elif avg >= 60:
        return "D"
    else:
        return "F"
df["Grade"] = df["Average"].apply(calculate_grade)
print("Student Grades:")
print(df[["Student_ID", "Name", "Average", "Grade"]])
df.to_csv(r"C:\\Users\\Rajku\\OneDrive\\Desktop\\Python\\updated_student_records.csv",
index=False)
print("Modified dataset saved as updated_student_records.csv")
```

Lab7.py - C:\Users\Rajku\OneDrive\Desktop\Python\Lab7.py (3.14.5)

File Edit Format Run Options Window Help

```python
#3
import pandas as pd
df = pd.read_csv(r"C:\\Users\\Rajku\\OneDrive\\Desktop\\Python\\student_rec
print("Student Records:")
print(df)
df["Average"] = (df["Math"] + df["Science"] + df["English"]) / 3
print("Average Marks of Each Student:")
print(df[["Student_ID", "Name", "Average"]])
specified_average = 80
above_average = df[df["Average"] > specified_average]
print("Students Scoring Above", specified_average, "Average:")
print(above_average[["Student_ID", "Name", "Average"]])
print("Math:", df["Math"].max())
print("Science:", df["Science"].max())
print("English:", df["English"].max())
print("Math:", df["Math"].min())
print("Science:", df["Science"].min())
print("English:", df["English"].min())
sorted_df = df.sort_values(by="Average", ascending=False)
print(sorted_df[["Student_ID", "Name", "Average"]])
def calculate_grade(avg):
    if avg >= 90:
        return "A"
    elif avg >= 80:
        return "B"
    elif avg >= 70:
        return "C"
    elif avg >= 60:
        return "D"
    else:
        return "F"
df["Grade"] = df["Average"].apply(calculate_grade)
print("Student Grades:")
print(df[["Student_ID", "Name", "Average", "Grade"]])
df.to_csv(r"C:\\Users\\Rajku\\OneDrive\\Desktop\\Python\\updated_student_rec
print("Modified dataset saved as updated_student_records.csv")
```

IDLE Shell 3.14.5

File Edit Shell Debug Options Window Help

```
Student Records:
   Student_ID     Name        Department  Math  Science  Engli
0         101    Alice  Computer Science    85       90
1         102      Bob       Mechanical    72       68
2         103  Charlie      Electronics    95       91
3         104    Diana            Civil    60       65
4         105    Ethan  Computer Science    78       82
5         106    Fiona       Mechanical    88       86
6         107   George      Electronics    55       58
7         108   Hannah            Civil    91       89
8         109      Ian  Computer Science    67       73
9         110    Julia       Mechanical    82       79
Average Marks of Each Student:
   Student_ID     Name    Average
0         101    Alice  87.666667
1         102      Bob  71.666667
2         103  Charlie  91.666667
3         104    Diana  65.000000
4         105    Ethan  80.000000
5         106    Fiona  88.000000
6         107   George  57.666667
7         108   Hannah  91.000000
8         109      Ian  69.666667
9         110    Julia  82.000000
Students Scoring Above 80 Average:
   Student_ID     Name    Average
0         101    Alice  87.666667
2         103  Charlie  91.666667
5         106    Fiona  88.000000
7         108   Hannah  91.000000
9         110    Julia  82.000000
Math: 95
Science: 91
English: 93
Math: 55
Science: 58
English: 60
   Student_ID     Name    Average
2         103  Charlie  91.666667
7         108   Hannah  91.000000
```